MINESnancy
ARTEM

Le Problème du Postier Chinois

Projet Optimisation dans les Graphes

Modélisation, Couplage Parfait et Algorithme d'Edmonds

Mohammed Chali

Mars 2026

Table des matières

1 Prérequis et Fondements Théoriques **3**

1.1 Concepts de base . . . 3

1.2 La théorie d'Euler . . . 3

2 Modélisation Mathématique et Preuve d'Optimalité **4**

2.1 Formulation Mathématique . . . 4

2.2 Preuve Combinatoire d'optimalité . . . 5

2.3 Remarque sur le coût total . . . 7

3 L'Algorithme d'Edmonds **7**

3.1 Étape 1 : Identification des sommets impairs . . . 8

3.2 Étape 2 : Construction du graphe G_p . . . 8

3.3 Étape 3 : Couplage Parfait de Poids Minimum . . . 8

3.3.1 Algorithme des Fleurs . . . 8

3.3.2 Minimisation des Coûts : Méthode Primale-Duale . . . 9

3.4 Étape 4 : Construction du multigraphe G' . . . 10

4 Extraction du Parcours : Cycle Eulérien **11**

5 Implémentation et Analyse de Complexité **12**

5.1 Choix de Représentation et Outils . . . 12

5.2 Analyse de la complexité temporelle . . . 12

5.2.1 Identification des sommets impairs . . . 12

5.2.2 Calcul des plus courts chemins . . . 12

5.2.3 Création du graphe complet G_p et Couplage optimal . . . 13

5.2.4 Extraction du Cycle Eulérien . . . 13

5.3 Bilan de l'Implémentation . . . 13

6 Applications Industrielles **13**

7 Conclusion **14**

8 Ouvertures et limites **14**

9 Application des résultats du rapport **15**

Références **15**

1 Prérequis et Fondements Théoriques

Avant de détailler la résolution exacte du problème du Postier Chinois, il est indispensable de définir les concepts fondamentaux de la théorie des graphes sur lesquels repose l'algorithme.

1.1 Concepts de base

Un graphe non orienté $G = (N, E)$ est défini par un ensemble de sommets N reliés par des arêtes appartenant à l'ensemble E. Dans un tel graphe, on appelle degré d'un sommet n, noté $d(n)$, le nombre d'arêtes incidentes à ce sommet.

Pour que le problème de routage ait un sens physique et puisse être résolu, le graphe étudié doit être connexe. Un graphe non orienté est dit connexe si, entre toute paire de sommets, il existe toujours au moins une chaîne (une suite finie d'arêtes) les reliant.

1.2 La théorie d'Euler

Le problème du Postier Chinois est **une généralisation du problème d'Euler** (illustré historiquement par les ponts de Königsberg). L'objectif du postier est de parcourir les rues d'une ville en passant au moins une fois par chaque rue et revenir au point d'origine, tout en minimisant la distance totale parcourue.

* La situation idéale correspond au cas où le graphe est *eulérien*. Un graphe est dit eulérien s'il admet un cycle eulérien, c'est-à-dire un parcours qui emprunte une fois et une seule chaque arête du graphe sans jamais lever le crayon et revient au point d'origine :

Théorème fondamental d'Euler

Un graphe non orienté possède un cycle eulérien si, et seulement si :

1. Il est **connexe**.
2. **Tous ses sommets ont un degré pair**.

Si **cette condition est respectée**, la solution au problème du Postier Chinois est simplement ce **cycle eulérien**, et la distance optimale est la somme exacte des pondérations de toutes les arêtes.

Cependant, dans le cas du problème du postier chinois, le graphe n'est pas forcément eulérien car il possède souvent des sommets de degré impair. La résolution du problème consistera alors dans un premier temps à rendre le graphe eulérien en dupliquant certaines arêtes, et à partir de ce graphe trouver le chemin optimal.

Pour corriger la parité des sommets, la modélisation s'appuie sur une règle mathématique de la théorie des graphes :

Lemme et Corollaire

Pour tout graphe non orienté $G = (N, E)$, chaque arête possédant exactement deux extrémités, la somme des degrés de tous les sommets du graphe est toujours égale au double du nombre total d'arêtes :

$$\sum_{n \in N} d(n) = 2|E|$$

Corollaire : En conséquence, **tout graphe possède un nombre toujours pair de sommets de degré impair**.

Remarque 1 : Intuitivement, le degré d'un nœud est le nombre de connexions à d'autres nœuds, ainsi le nombre d'arêtes reliant ce nœud aux autres. Ainsi en sommant tous les degrés, les arêtes sont comptées deux fois.

Remarque 2 : C'est cette propriété cruciale qui garantit que l'on pourra toujours associer ces sommets impairs par paires (sans en laisser un seul isolé) lors de l'étape de couplage des nœuds explicitée par la suite.

2 Modélisation Mathématique et Preuve d'Optimalité

Cette partie démontre rigoureusement pourquoi **la résolution du problème du Postier Chinois se réduit à la recherche d'un couplage parfait de poids minimum**, en s'appuyant sur les travaux fondateurs d'Edmonds et Johnson [2].

2.1 Formulation Mathématique

Soit un graphe connexe $G = (N, E)$. L'objectif est d'obtenir un graphe modifié G' admettant un cycle eulérien. Soit $1 + x_e$ le nombre de fois que l'arête e est présente dans le parcours. La variable entière $x_e \geq 0$ représente le nombre de **copies supplémentaires de l'arête** e ajoutées au graphe initial **pour former G'**.

Pour décrire ce problème, on définit la matrice d'incidence nœud-arête (a_{ne}) telle que $a_{ne} = 1$ si l'arête e est incidente au nœud n, et 0 sinon. La condition pour que le graphe G' admette un cycle eulérien est que chaque nœud soit de degré pair, ce qui s'écrit :

$$\sum_{e \in E} a_{ne}(1 + x_e) \equiv 0 \pmod 2 \tag{1}$$

avec $\sum_{e \in E} a_{ne}(1 + x_e)$, le nouveau degré du nœud n dans le graphe G'.

Cette congruence peut être réécrite de la manière suivante :

$$\sum_{e \in E} a_{ne} x_e \equiv \sum_{e \in E} a_{ne} \pmod 2 \tag{2}$$

avec $\sum_{e \in E} a_{ne}$, le degré initial du nœud n dans le graphe G et $\sum_{e \in E} a_{ne} x_e$ le nombre de degrés supplémentaires à ajouter au nœud n pour former G'.

Remarque : Ainsi, nous voulons que le nombre de degrés supplémentaires à ajouter à un nœud n puisse faire en sorte que le nouveau degré dans G' soit pair.

On pose b_n tel que $b_n \equiv \sum_{e \in E} a_{ne} \pmod 2$. Ainsi, $b_n = 0$ si le degré initial du nœud n est pair, et $b_n = 1$ s'il est impair. Le problème du Postier Chinois se modélise alors par le programme linéaire en nombres entiers suivant :

Programme d'Optimisation du Postier Chinois

Minimiser la fonction objectif :

$$z = \sum_{e \in E} c_e x_e \tag{3}$$

où $c_e \geq 0$ est la longueur de l'arête e.

Sous les contraintes :

$$\sum_{e \in E} a_{ne} x_e - 2w_n = b_n, \qquad \forall n \in N \tag{4}$$

$$x_e \geq 0 \text{ et entier}, \quad \forall e \in E \tag{5}$$

$$w_n \geq 0 \text{ et entier}, \quad \forall n \in N \tag{6}$$

La variable auxiliaire w_n permet de transformer la congruence modulo 2 en une égalité stricte, garantissant que la somme des arêtes supplémentaires ajoutées compense exactement la parité initiale b_n du sommet.

2.2 Preuve Combinatoire d'optimalité

Edmonds et Johnson prouvent qu'une solution optimale à ce système se réduit finalement à des chemins reliant les nœuds impairs. La démonstration s'articule en trois étapes logiques :

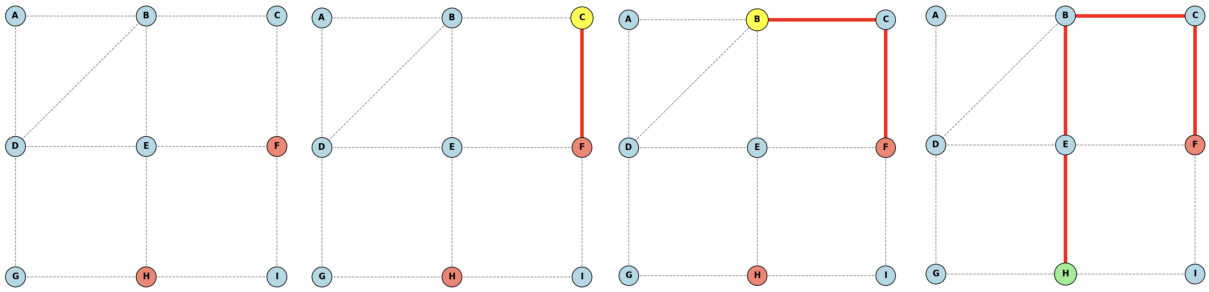
- **Simplification des variables ($x_e \in \{0, 1\}$) :** Dans une solution optimale, on peut supposer que x_e vaut soit zéro, soit un. En effet, par absurde, s'il y avait une arête dupliquée plus d'une fois (c'est-à-dire $x_e \geq 2$), on pourrait simplement réduire cette valeur de deux, puisque :
  - Retirer 2 passages diminue le degré des nœuds adjacents de 2. Or, soustraire 2 ne modifie jamais la parité d'un nombre. **Le graphe conserve ses propriétés eulériennes.**
  - Éviter ces deux passages permet d'économiser une distance de $2 \times c_e$. Puisque les distances sont positives ou nulles ($c_e \geq 0$), le coût total z diminuerait, ce qui contredit l'optimalité.
- **Décomposition en chemins simples :** La structure des arêtes dupliquées (celles où $x_e = 1$) n'est pas aléatoire. Puisque ces arêtes doivent corriger la parité du graphe, elles obéissent à deux règles strictes :

— Sur un nœud **initialement pair**, il doit y avoir un nombre *pair* d'arêtes supplémentaires. L'une sert d'entrée, l'autre de sortie.

— Sur un nœud **initialement impair**, il doit y avoir un nombre *impair* d'arêtes supplémentaires. Ce nœud agit donc comme une "extrémité" (un point de départ ou d'arrivée).

Si l'on commence un parcours à partir d'un nœud impair en suivant uniquement ces arêtes dupliquées ($x_e = 1$), on va simplement traverser (entrer et sortir) les nœuds pairs intermédiaires sans jamais pouvoir s'y bloquer. Le parcours s'arrêtera obligatoirement lorsqu'on rencontrera un autre nœud impair.

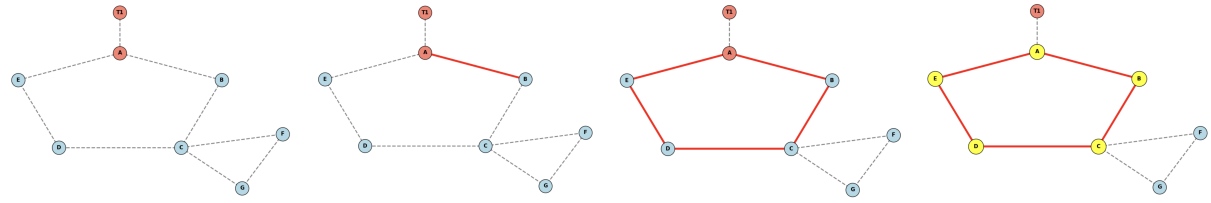
En effaçant ce chemin nouvellement trouvé et en répétant l'opération, on démontre que l'**ensemble des arêtes pour lesquelles** $x_e = 1$ forment des séries de **chemins simples connectant les nœuds impairs par des nœuds pairs.**



(a) F et H impairs (b) Ajout arête, C impair (c) Ajout arête, B impair (d) État final

FIGURE 1 – Exemple de correction de la parité ($x_e = 1$).

— **Élimination des cycles superflus :** Après avoir connecté tous les nœuds impairs, les seules arêtes restantes ayant $x_e = 1$ forment nécessairement des cycles (tours), car elles ont un degré pair à chaque nœud. Or, **ces arêtes peuvent toutes être réduites à** $x_e = 0$ sans violer les contraintes de parité et sans augmenter le coût z (puisque $c_e \geq 0$).



(a) A est impair (b) Ajout arête, B impair (c) Formation d'un cycle (d) Deuxième cycle

FIGURE 2 – Exemple de formation de cycle ($x_e = 1$).

Nous avons ainsi démontré qu'une solution optimale aux équations (4), (5) et (6) se décompose **uniquement** en une collection de chemins simples reliant les nœuds impairs par paires.

Le problème à présent est de choisir *quelles* paires former, et *quels* chemins emprunter pour minimiser le coût total z.

Pour garantir l'obtention de la valeur minimale absolue, la démarche d'Edmonds et Johnson se modélise en trois étapes :

1. **Plus courts chemins :** Pour toute paire de nœuds impairs (i, j) que l'on déciderait de

relier, le moyen le moins coûteux de le faire est, par définition, d'emprunter le plus court chemin entre i et j dans le graphe d'origine G. On calcule donc ces plus courts chemins (par exemple via l'algorithme de Dijkstra) pour absolument toutes les paires possibles de nœuds impairs.

2. **Création du graphe auxiliaire complet (G_p) :** On construit ensuite un nouveau graphe, noté G_p. Ses sommets sont *uniquement* les nœuds impairs du graphe d'origine G. On relie chaque sommet de G_p à tous les autres pour former un graphe complet. L'astuce fondamentale est d'attribuer à chaque arête (i, j) de ce graphe un poids égal au coût du plus court chemin calculé à l'étape précédente.
3. **Optimisation globale :** Le problème initial se réduit alors à trouver un **Couplage Parfait de Poids Minimum** (1-matching) sur ce graphe complet G_p. Un algorithme de couplage va sélectionner un ensemble d'arêtes de G_p tel que chaque nœud impair soit apparié exactement une fois, et de sorte que la somme des poids de ces arêtes soit la plus petite possible.

2.3 Remarque sur le coût total

Le coût total d'un parcours du Postier Chinois (Z_{total}) se divise en deux parties :

$$Z_{total} = \text{Coût de base} + \text{Coût des détours (Pénalité)} \tag{7}$$

— **Le coût de base :** Le postier doit emprunter chaque arête du graphe G au moins une fois. La somme des poids de toutes les arêtes initiales ($\sum_{e \in E} c_e$) est donc une constante incompressible. Nous ne pouvons pas faire moins.
— **La pénalité :** À cause des nœuds impairs, le postier est forcé de traverser certaines arêtes une deuxième fois. C'est cette somme d'arêtes supplémentaires ($\sum_{e \in E} c_e x_e$) qu'il faut minimiser.

Le couplage parfait de poids minimum calculé sur le graphe G_p fournit très exactement la **pénalité minimale absolue**. Les arêtes sélectionnées par le couplage correspondent aux plus courts chemins reliant les nœuds impairs. Ce sont précisément ces chemins que le postier devra emprunter une deuxième fois.

À ce stade, nous n'avons pas résolu le problème du postier chinois, nous avons seulement calculé le coût total de la solution optimale et déterminé les arêtes qui devront être empruntées deux fois.

3 L'Algorithme d'Edmonds

Détaillons à présent l'algorithme de résolution de ce problème de couplage parfait de poids minimum.

3.1 Étape 1 : Identification des sommets impairs

La première étape est une simple inspection du graphe d'origine $G = (N, E)$. L'algorithme parcourt chaque nœud et calcule son degré (le nombre d'arêtes qui y sont connectées). On isole et on stocke dans une liste O tous les nœuds dont le degré est impair. D'après le Lemme vu à la Partie 1, nous avons la garantie absolue que la taille de cette liste O est un nombre pair. Il n'y aura donc aucun nœud impair *laissé* sans couplage.

3.2 Étape 2 : Construction du graphe G_p

Pour savoir quelles paires de nœuds impairs associer, il faut d'abord connaître le *coût* de chaque association possible.

- **Calcul des distances :** On applique un algorithme de recherche de plus courts chemins (comme l'algorithme de Dijkstra) entre toutes les paires possibles de nœuds présents dans l'ensemble O.
- **Création du graphe virtuel (G_p) :** On construit un tout nouveau graphe G_p. Ses sommets sont uniquement les nœuds de l'ensemble O. On relie absolument tous ces sommets entre eux pour former un *graphe complet*.
- **Pondération :** Chaque arête virtuelle reliant un nœud i à un nœud j dans G_p reçoit un poids égal à la distance du plus court chemin entre i et j calculée dans le graphe d'origine G.

3.3 Étape 3 : Couplage Parfait de Poids Minimum

3.3.1 Algorithme des Fleurs

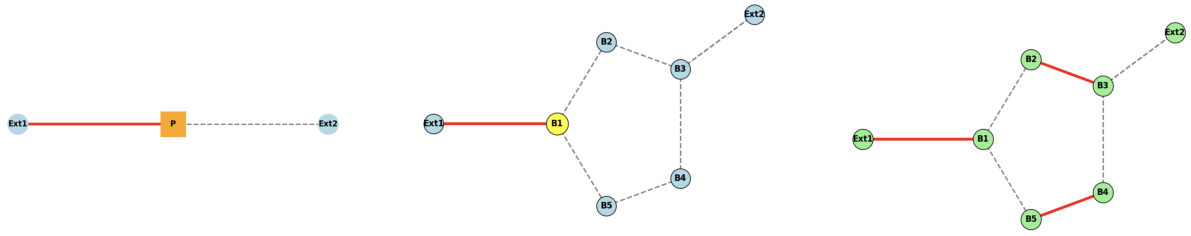
Contrairement à un problème d'affectation classique (ouvriers/tâches) qui utilise un graphe biparti, notre graphe G_p est un graphe général (n'importe quel nœud peut être couplé avec n'importe quel autre). Cela rend le problème plus complexe à cause de l'existence de **cycles de longueur impaire**. C'est ici qu'intervient l'**Algorithme des Fleurs (Blossom Algorithm)** inventé par Jack Edmonds.

L'algorithme cherche à agrandir un couplage partiel en trouvant des **chemins augmentants** (des chemins qui alternent entre des arêtes non-couplées et couplées, et qui relient deux sommets encore libres).

Cependant, dans un graphe non-biparti, la recherche de ces chemins peut tourner en rond et échouer si elle rencontre un cycle de longueur impaire. Edmonds a résolu ce problème par un mécanisme de *Contraction / Expansion* :

Algorithme des Fleurs

1. **Découverte de la Fleur (Blossom) :** Lorsque l'algorithme d'exploration détecte un cycle de longueur impaire formé d'arêtes alternées, ce cycle est appelé une "Fleur".
2. **Contraction (Shrinking) :** L'astuce consiste à fusionner mathématiquement tous les sommets de ce cycle impair pour n'en faire plus qu'un seul "pseudo-sommet". Le graphe devient alors plus petit et plus simple. L'algorithme relance sa recherche de chemin augmentant sur ce graphe contracté.
3. **Expansion (Expanding) :** Une fois qu'un couplage optimal est trouvé sur le graphe simplifié, l'algorithme fait le chemin inverse. Il "épanouit" (restaure) les pseudo-sommets pour retrouver les cycles d'origine. Puisque le cycle est de longueur impaire, il y a toujours une façon unique et parfaite de réarranger les arêtes du couplage à l'intérieur du cycle pour que chaque nœud trouve son partenaire exact.



(a) Le point P est une fleur (b) Expansion (c) Couplage

FIGURE 3 – Illustration de la phase d'expansion ($x_e = 1$).

Grâce à ce mécanisme, l'algorithme permet de résoudre le problème des cycles impairs dans G_p.

3.3.2 Minimisation des Coûts : Méthode Primale-Duale

Une fois le problème des cycles impairs résolu par la contraction en pseudo-sommets, il reste à garantir que le couplage parfait obtenu est de **poids minimum**. Pour ce faire, l'algorithme d'Edmonds n'explore pas les arêtes au hasard : il utilise la théorie de la dualité en maintenant simultanément une solution primale (le couplage) et une solution duale (les potentiels des sommets).

1. Variables Duales et Écarts

Soit le graphe $G_p = (N, E)$ où chaque arête $e = (u, v)$ possède un poids $c_e \geq 0$. L'algorithme associe à chaque sommet $v \in N$ une variable mathématique appelée **potentiel dual**, notée y_v. (Dans le cas d'un pseudo-sommet contracté, celui-ci reçoit également sa propre variable duale $y_S \geq 0$).

Pour toute arête $e = (u, v)$ du graphe, on définit son **écart** (ou *slack*), noté Δ_e, par l'équation suivante :

$$\Delta_e = c_e - y_u - y_v \tag{8}$$

La contrainte duale stricte impose que cet écart soit toujours positif ou nul ($\Delta_e \geq 0$), ce qui signifie que la somme des potentiels des deux extrémités ne peut jamais dépasser le poids de l'arête ($y_u + y_v \leq c_e$).

2. Le Sous-graphe d'Égalité

La règle fondamentale de l'algorithme (basée sur le théorème des écarts complémentaires) est la suivante : **L'algorithme n'est autorisé à construire son arbre alterné et à sélectionner des arêtes pour le couplage que si leur écart est strictement nul ($\Delta_e = 0$).**

Une arête vérifiant $y_u + y_v = c_e$ est dite **saturée**. L'ensemble de ces arêtes forme le *sous-graphe d'égalité*. L'algorithme **cherche des chemins augmentants exclusivement dans ce sous-graphe.**

3. Blocage et Mise à Jour Duale

Que se passe-t-il si l'arbre alterné se bloque parce qu'il n'y a plus aucune arête saturée disponible pour atteindre un sommet libre ? C'est ici que l'optimisation des poids s'opère :

1. L'algorithme analyse toutes les arêtes non-saturées ($\Delta_e > 0$) qui relient un sommet de l'arbre classé dans l'ensemble **P** (sommet à distance paire de la racine) à un sommet extérieur à l'arbre.
2. Il calcule la valeur δ, définie comme le **plus petit écart** parmi toutes ces arêtes : $\delta = \min\{\Delta_e\}$
3. L'algorithme effectue alors une **mise à jour duale** :
   — Il augmente le potentiel y_v de tous les sommets **P** de l'arbre de la valeur $+\delta$.
   — Il diminue le potentiel y_v de tous les sommets **I** (impairs) de l'arbre de la valeur $-\delta$.

4. Garantie d'Optimalité

Cette mise à jour duale a une conséquence mathématique immédiate : l'écart Δ_e de l'arête qui possédait le minimum δ tombe exactement à zéro. **Une nouvelle arête vient d'être saturée** et s'ajoute au sous-graphe d'égalité. L'arbre alterné peut reprendre son exploration.

Puisque l'algorithme ne fait croître les potentiels y_v que de la valeur minimale stricte nécessaire pour débloquer de nouvelles arêtes, il garantit de n'emprunter que les chemins globalement les moins coûteux. Lorsqu'un couplage parfait est finalement trouvé dans ce sous-graphe d'égalité, le théorème de la dualité forte assure que la somme des poids du couplage ($\sum_{e \in E} c_e$) est exactement égale à la somme des potentiels duaux ($\sum_{v \in N} y_v$). Ce couplage assure donc mathématiquement le coût minimal.

3.4 Étape 4 : Construction du multigraphe G'

L'étape finale consiste à ramener ce couplage virtuel dans le monde réel. Pour chaque arête virtuelle (i, j) qui a été sélectionnée par le couplage optimal dans G_p, on retourne regarder le chemin physique qu'elle représente dans le graphe d'origine G.

On **duplique** alors toutes les arêtes de ce chemin physique dans G (ce qui revient à assigner $x_e = 1$ aux variables de notre modèle mathématique). En effectuant cette opération pour toutes les paires du couplage, tous les sommets initialement impairs de G reçoivent un nombre impair d'arêtes supplémentaires, devenant ainsi de degré pair.

Le graphe d'origine G a été transformé en un multigraphe G' où **absolument tous les sommets sont de degré pair**. Le graphe est désormais **eulérien**. Nous pouvons maintenant chercher le parcours (le cycle eulérien).

4 Extraction du Parcours : Cycle Eulérien

À l'issue de l'étape de couplage et de duplication, notre graphe G' est devenu un graphe strictement eulérien : il est connexe et tous ses sommets possèdent un degré pair. Le théorème d'Euler nous garantit mathématiquement qu'il existe un cycle empruntant chaque arête exactement une fois puis revenant au point d'origine.

L'article d'Edmonds et Johnson détaille plusieurs méthodes pour extraire ce parcours, notamment l'algorithme des paires d'extrémités (*End-pairing algorithm*) et l'algorithme du nœud suivant (*Next-node algorithm*) qui ne seront pas explicités. L'algorithme que nous utiliserons repose sur le principe de construction par sous-cycles (souvent attribué à Carl Hierholzer).

Algorithme de Construction du Cycle Eulérien

Le principe consiste à marcher au hasard dans le graphe, en marquant les rues traversées pour ne jamais les reprendre, jusqu'à être naturellement bloqué.

1. **Premier sous-cycle :** Le postier part d'un sommet de départ (la poste) et emprunte une arête non visitée au hasard. Puisque chaque sommet a un nombre pair d'arêtes, chaque fois qu'il entre dans une intersection, il y a forcément une porte de sortie non visitée. Le seul moment où il se retrouvera bloqué, c'est lorsqu'il reviendra à son *point de départ* et que toutes les arêtes de ce point auront été visitées. Il a ainsi tracé un premier "sous-cycle" fermé.
2. **Vérification et relance :** On regarde si toutes les arêtes de la ville ont été visitées. Si oui, le trajet est terminé. Si non, le graphe étant connexe, il existe forcément au moins un sommet sur notre sous-cycle actuel qui possède encore des arêtes non visitées.
3. **Nouveau sous-cycle et Fusion :** On prend ce sommet comme nouveau point de départ, et on relance une marche au hasard. On obtient un deuxième sous-cycle fermé. La prochaine étape consiste alors à **fusionner** ces deux boucles : le postier commence son premier cycle, mais lorsqu'il arrive à l'intersection commune, il parcourt entièrement le deuxième cycle avant de terminer le premier.
4. **Construction par itération :** Ces opérations sont répétées jusqu'à ne plus avoir d'arêtes non visitées. Ce parcours fermé correspond alors à la solution optimale.

En répétant cette opération (trouver des sous-cycles sur les arêtes restantes et les insérer dans le trajet principal), l'algorithme *inclut* progressivement toutes les arêtes du graphe dans le trajet. L'ordre final des sommets visités constitue le trajet optimal du Postier Chinois.

5 Implémentation et Analyse de Complexité

Afin de valider notre modèle théorique, nous avons procédé à l'implémentation complète de l'algorithme du Postier Chinois. Cette section détaille nos choix techniques, les structures de données utilisées, et propose une analyse de la complexité temporelle des algorithmes et fonctions utilisés.

5.1 Choix de Représentation et Outils

Le développement s'est appuyé sur **NetworkX** [4], la bibliothèque Python de référence pour la théorie des graphes.

- **Le Graphe Initial (G) :** L'utilisation d'une matrice d'adjacence aurait entraîné un "gaspillage" mémoire en $\mathcal{O}(|N|^2)$. Nous avons donc utilisé la structure `nx.Graph()`, qui implémente un dictionnaire de dictionnaires (une version plus optimisée des listes d'adjacence). Cette approche garantit une utilisation mémoire en $\mathcal{O}(|N| + |E|)$ et un accès en temps constant $\mathcal{O}(1)$ à l'arête reliant deux nœuds adjacents.
- **Le Graphe Eulérien (G') :** Puisque la résolution du problème nécessite la duplication de certaines arêtes (rues parcourues deux fois), le graphe d'origine G ne suffisait plus (un graphe simple n'autorisant qu'une seule arête entre deux sommets). Nous avons utilisé une structure de **Multigraphe** (`nx.MultiGraph()`).

5.2 Analyse de la complexité temporelle

L'algorithme se décompose en cinq phases distinctes. Soit $|N|$ le nombre total de sommets, $|E|$ le nombre total d'arêtes du graphe de départ, et $|I|$ le nombre de sommets de degré impair.

5.2.1 Identification des sommets impairs

Le calcul du degré s'est effectué via la méthode `G.degree()`.

- **Complexité :** $\mathcal{O}(|N|)$. Le parcours des nœuds pour isoler la liste `odd_nodes` est strictement linéaire.

5.2.2 Calcul des plus courts chemins

Pour évaluer le coût de couplage, nous avons généré toutes les paires possibles de nœuds impairs via `itertools.combinations`, soit $\frac{|I|(|I|-1)}{2}$ paires. Pour chacune, la fonction `nx.shortest_path_length` (basée sur l'algorithme de Dijkstra) a été appelée.

— **Complexité :** Un appel à Dijkstra sur notre structure de données avec coûte $\mathcal{O}(|E| + |N|\log|N|)$ [3]. Puisque nous lançons la recherche depuis les $|I|$ nœuds impairs, la complexité de cette étape est de $\mathcal{O}(|I| \cdot (|E| + |N|\log|N|))$.

5.2.3 Création du graphe complet G_p et Couplage optimal

Nous avons instancié un graphe complet G_p contenant les $|I|$ sommets. Étant donné que NetworkX ne propose qu'une fonction de maximisation (`nx.max_weight_matching`), nous avons inversé les poids ($Poids_{inverse} = Poids_{max} - Poids_{initial}$) pour forcer la minimisation.

— L'inversion des poids parcourt les arêtes de G_p en $\mathcal{O}(|I|^2)$.

— Une version améliorée de l'algorithme de couplage parfait d'Edmonds est ensuite exécuté par NetworkX (Zvi Galil) [4].

— **Complexité :** La recherche du couplage sur un graphe de $|I|$ sommets s'exécute en $\mathcal{O}(|I|^3)$. C'est théoriquement l'étape la plus coûteuse en terme de complexité.

5.2.4 Extraction du Cycle Eulérien

Le multigraphe G' possédant désormais exclusivement des degrés pairs, nous extrayons le cycle optimal (l'ordre de parcours des arêtes) en un seul passage, typiquement via l'algorithme de Hierholzer.

— **Complexité :** $\mathcal{O}(|E'|)$ où $|E'|$ représente le nombre total d'arêtes final ($|E'| = |E|$ + doublons). L'extraction est donc strictement proportionnelle à la taille finale du parcours.

5.3 Bilan de l'Implémentation

Les différentes étapes de l'algorithme donnent une complexité temporelle de :

$$\mathcal{O}_{total} = \mathcal{O}\Big(|I| \cdot (|E| + |N|\log|N|) + |I|^3\Big) \tag{9}$$

En transformant un problème de routage en un sous-problème de couplage, cet algorithme fournit **un parcours exact et optimal** en un **temps polynomial**, démontrant l'appartenance du problème du Postier Chinois à la classe de complexité P.

6 Applications Industrielles

Bien que formulé initialement en 1962 par le mathématicien Kwan Mei-Ko [1] pour optimiser la tournée d'un facteur, le problème du Postier Chinois trouve aujourd'hui de très nombreuses applications dans la logistique urbaine et la recherche opérationnelle moderne.

Contrairement au problème du Voyageur de Commerce (où l'objectif est de visiter des *points* précis comme des villes ou des clients), le Postier Chinois s'applique lorsqu'il est nécessaire

de traiter un **réseau physique** (les arêtes). On retrouve ce besoin dans plusieurs secteurs industriels :

— **Nettoyage urbain et déneigement :** Le ramassage des ordures ménagères, le balayage des rues ou encore le déneigement en période d'hiver. Le véhicule doit impérativement parcourir toutes les rues d'un secteur. Minimiser la distance totale permet de réduire la consommation de carburant et les émissions de gaz à effet de serre par exemple.
— **Inspection et maintenance des infrastructures :** La vérification des lignes électriques ou la détection de fuites sur un réseau de canalisations en optimisant le déplacement.
— **Sécurité :** L'optimisation des rondes de sécurité (agents de surveillance, police) dans un quartier ou un complexe industriel.

Ces applications réelles justifient l'importance stratégique de disposer d'un algorithme exact s'exécutant en temps polynomial.

7 Conclusion

Le Problème du Postier Chinois illustre la puissance de l'optimisation dans les graphes. Plutôt que de chercher à tracer un itinéraire complexe par essais et erreurs, la méthode d'Edmonds et Johnson transforme un problème de *routage* en un problème de *parité*, qui lui-même se résout par un problème de *couplage* sur un graphe.

Cette approche garantie :

— **Une solution optimale :** Comme démontré, l'algorithme d'Edmonds garantit de trouver le cout le plus faible possible.
— **Une efficacité algorithmique :** Contrairement au problème du Voyageur de Commerce (TSP) qui est NP-difficile, toutes les étapes de cet algorithme (calcul des plus courts chemins par Dijkstra, couplage par l'algorithme d'Edmonds, et extraction du cycle eulérien) s'exécutent en un temps polynomial. Le problème du Postier Chinois appartient donc à la classe de complexité P.

8 Ouvertures et limites

Si le modèle standard est aujourd'hui parfaitement maîtrisé, la réalité opérationnelle de la logistique urbaine impose de nouvelles contraintes. En effet, l'article d'Edmonds et Johnson évoque également le cas des graphes orientés (rues à sens unique), qui se résout efficacement par des algorithmes de flot maximum.

Cependant, le **Problème du Postier Chinois Mixte** (un réseau urbain contenant *à la fois* des rues à double sens et des rues à sens unique), une variante très réaliste pour le ramassage des ordures ou le déneigement, a été prouvée NP-difficile. La résolution exacte laisse alors place au développement de métaheuristiques.

9 Application des résultats du rapport

L'application concrète des algorithmes expliqués dans ce rapport se trouve dans la partie Annexe.

Références

[1] Kwan Mei-Ko. *Graphic Programming Using Odd or Even Points*. Chinese Mathematics, 1962.
(L'article historique qui a introduit le Problème du Postier Chinois pour la première fois).

[2] Jack Edmonds et Ellis L. Johnson. *Matching, Euler tours and the Chinese postman*. Mathematical Programming, 1973.
(L'article mathématique fondateur détaillant l'algorithme des Fleurs et la méthode primale-duale).

[3] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, et Clifford Stein. *Algorithmique (Introduction to Algorithms)*. Dunod, 3ème édition, 2009.
(Ouvrage de référence pour l'analyse de complexité et les algorithmes de plus courts chemins comme Dijkstra).

[4] Aric A. Hagberg, Daniel A. Schult et Pieter J. Swart. *Exploring Network Structure, Dynamics, and Function using NetworkX*. Proceedings of the 7th Python in Science Conference (SciPy), 2008. Documentation en ligne : https://networkx.org/
(La référence officielle pour justifier les choix d'implémentation en Python).